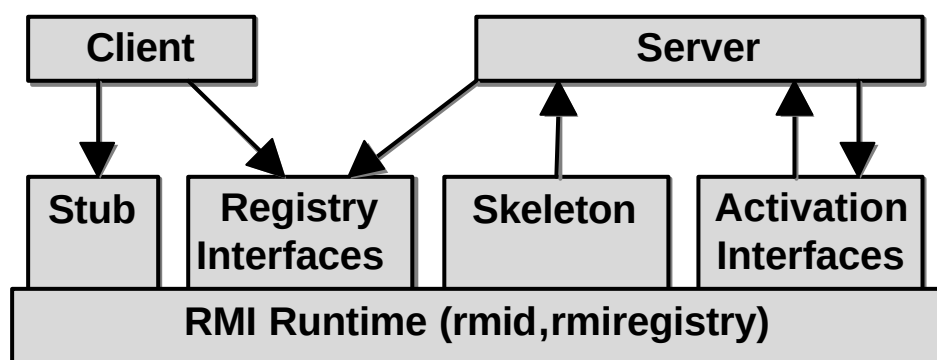


Java RMI (2)

RMI architecture



Parameter Passing

- Atomic types are passed by value
- Remote objects are passed by reference
- Non-Remote objects are passed by value

Naming

- Clients find remote services by using a naming or directory service.
- Naming & directory services are run on known hosts and ports
- RMI can use a variety of different services.
 - Java Naming and Directory Service (JNDI)
 - rmiregistry

rmiregistry

- Runs on each machine that hosts remote service objects and accepts queries for services
- Default port is 1099.
- On the server
 - RemoteObject is exported
 - RemoteObject is registered in registry using `Naming.rebind(String name, Remote obj);`

rmiregistry

- On the client
 - Remote objects are located using `Naming.lookup(String url)` where the URL is specified in the format:
`rmi://<host_name> [:<name_service_port>] /<service_name>`
- `Naming.lookup(String url)` returns a reference to an object that implements the Remote interface. This needs to be cast (“narrowed”) to the relevant more specialised type of the object.

Remote Objects as parameters

- So far we have looked at fairly simple client/server scenarios in which clients gain a reference to a server object via a naming service and then make remote object method calls upon the server object.
- As this is a fairly simple naming service it is often used just as a “bootstrap” service which starts up a system.
- From then on objects can create more remote objects and pass references to them as parameters or return types of remote interface methods.

Passing Remote Objects

- Remote Objects are commonly passed as parameters and return types.
- Example usage:
 - Callbacks
 - Factory classes that create Remote references
- Reminder: Remote Objects are passed by reference.
- When Remote exported objects are passed to a client, RMI substitutes the reference with that of the Remote proxy (stub).

Callbacks

- In many cases, applications require more complex 2-way interactions.
- Servers may wish to make calls back to the client (known as a callback).
 - Error or problem reporting
 - Periodic updating & progress reports
- In OO programs the role of clients and servers are not always clear cut.
- They often operate in a peer-to-peer manner. At different stages an object may either act as a server or as a client.

Callback - Howto

- How do you create a callback?
 - Make your client into a server!
- Make your client implement a Remote interface.
 - Define a client remote interface
- Make it available as a Server (export your client interface as a remote object)
 - extend `UnicastRemoteObject`
 - or use `UnicastRemoteObject.exportObject(Remote)`
- Pass a client remote reference to the server. The server can then use this reference to make calls on the client.

Remote Interfaces

```
import java.rmi.*;

public interface DateCBCClientInterface extends Remote
{
    public void print(String output) throws
        RemoteException;
}

public interface DateCBServerInterface extends Remote
{
    public void getDate(DateCBCClientInterface client)
        throws RemoteException;
}
```

Callback - Client Code

```
public class DateCBCClient extends UnicastRemoteObject implements
DateCBCClientInterface
{
    DateCBCClient() throws RemoteException
    {
        super();
    }

    // the remote client interface method
    public void print(String output)
    {
        System.out.println("client side printing");
        System.out.println(output);
    }
}
```

DateCBClient (cont.)

```
public static void main(String[] args)
{
    System.setSecurityManager(new RMISecurityManager());
    try {
        System.out.println("Security Manager loaded");
        String url = "///localhost/DATECB-SERVER";
        DateCBServerInterface remoteObject =
            (DateCBServerInterface)Naming.lookup(url);
        DateCBClient localclient = new DateCBClient();
        remoteObject.getDate(localclient);
    }
    catch (RemoteException exc){...}
    catch (java.net.MalformedURLException exc) {...}
    catch (java.rmi.NotBoundException exc) {...}
}
} // end DateCBClient
```

DateCBServerImpl

```
public class DateCBServerImpl extends
UnicastRemoteObject implements DateCBServerInterface
{
    DateCBServerImpl() throws RemoteException
    {
        super();
    }

    public void getDate(DateCBClientInterface client)
    throws RemoteException
    {
        DateCBClientInterface remoteClient = client;
        remoteClient.print("calling print on client");
        remoteClient.print(new Date().toString());
    }
}
```

DateCBServer

```
public class DateCBServer
{
    public static void main(String args[])
    {
        try {
            System.setSecurityManager(new RMISecurityManager());
            DateCBServerImpl dateServer = new DateCBServerImpl();
            Naming.rebind("DATECB-SERVER" , dateServer);
        }
        catch (java.net.MalformedURLException me) {
            System.out.println("Malformed URL: " + me.toString());
        }
        catch (RemoteException re) {
            System.out.println("Remote exception: " +
re.toString());
        }
    }
}
```

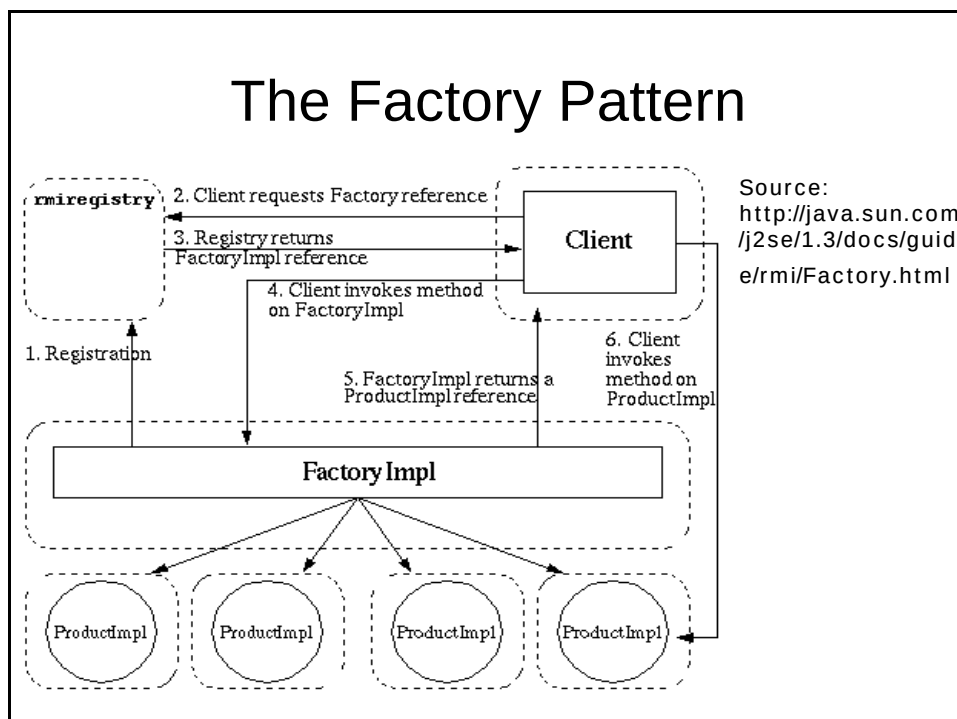
Design Patterns

- Design patterns are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context

"Design Patterns: Elements of Reusable Object-Oriented Software", Gamma et al.

The Factory Pattern

- The Factory Pattern
 - a factory pattern implementation is useful when you need one object to control the creation of and/or access to other objects.
- Factory classes in RMI can assist in reducing the number of objects that you need to register with the RMI registry.



The Factory Pattern

- There are two remote interfaces that the client understands, Factory and Product
- The FactoryImpl implements the Factory interface and the ProductImpl implements the Product interface
- Instead of asking the registry for a Product reference you ask the Factory for a reference to a Factory object.

Factory Pattern - Uses

- We can use the one Factory to return different subtypes of an interface (polymorphism).
- We reduce the coupling to a Naming service.
- We can implement different Factory implementation strategies
 - object pooling, load balancing
 - singleton factory
- The pattern can be used for non-remote objects as well.
- Uses: BankAccount, Library, ChatRoom?..

Activation

- So far we have dealt with servers that have been started manually before clients could gain a reference and make remote calls.
- This requires servers to be up and running regardless of whether they are required or not.
- As of JDK1.2 it is possible to create servers that run on demand.

Object Activation

- **Activatable**
 - an abstract class that provides default behaviour
- **rmid**
 - activation system daemon that allows objects to be registered and activated in a Java virtual machine (JVM).

Object Activation

- Create an implementation class that extends `Activatable`
- Create a Setup class that registers your implementation class with `rmiid`
 - Create an `ActivationGroup` instance
 - Create an `ActivationDesc` instance
 - Declare an instance of your remote interface and register with `rmiid`
- Start a registry and `rmiid` and run Setup class

Activation - Implementation class

- Extend your class from `java.rmi.activation.Activatable`
(instead of `UnicastRemoteObject`)
- Declare a two-argument constructor in the implementation class

```
public ActiveDateImpl(ActivationID id,
    MarshalledObject data) throws RemoteException {
    super(id, 0);
}
```
- Implement the remote interface method(s)

Activation - Setup class (Server)

- Make the appropriate imports
- Install a SecurityManager
- Create an ActivationGroup instance
- Create an ActivationDesc instance
- Declare an instance of your remote interface and register with rmid
- Bind the stub to a name in the rmiregistry
- Quit the setup application

Activation - Setup class (Server)

- Make the appropriate imports

```
import java.rmi.*;  
import java.rmi.activation.*  
import java.util.Properties;
```

- Install a SecurityManager

```
System.setSecurityManager(new  
    RMISecurityManager());
```

Create an ActivationGroup instance

- ActivationGroupDesc provides all the information that rmid will require to activate the object

```
// Because of the Java 2 security model, a security policy
// should be specified for the ActivationGroup VM.
Properties props = new Properties();
props.put("java.security.policy",
    "/home/bruce/activedate/policy.all");
ActivationGroupDesc.CommandEnvironment ace = null;
ActivationGroupDesc exampleGroup = new
    ActivationGroupDesc(props, ace);

// Once the ActivationGroupDesc has been created, register it
// with the activation system to obtain its ID
ActivationGroupID agi =
    ActivationGroup.getSystem().registerGroup(exampleGroup);

ActivationGroup.createGroup(agi, exampleGroup, 0);
```

Create an ActivationDesc instance

```
// The "location" String specifies a URL from where the class
// definition will come when this object is requested
// (activated).
// Don't forget the trailing slash at the end of the URL
// or your classes won't be found.
String location = "file:/home/bruce/activedate/";

// Create the rest of the parameters that will be passed to
// the ActivationDesc constructor
MarshaledObject data = null;

// The second argument to the ActivationDesc constructor will
// be used
// to uniquely identify this class; it's location is relative
// to the
// URL-formatted String, location.
ActivationDesc desc = new ActivationDesc
    ("ActiveDateServerImpl", location, data);
```

Registering your implementation

- Declare an instance of your remote interface and register with rmid

```
ActiveDateInterface myActiveDate =  
    (ActiveDateInterface)Activatable.register(d  
    esc);
```

- Bind the stub to a name in the rmiregistry
`Naming.rebind("ACTIVE_DATE", myActiveDate);`
`System.out.println("Exported ACTIVE_DATE");`
- Quit the setup application
`System.exit(0);`

Running it

- Start a registry and rmid and run Setup class
 - Compile all classes eg. `javac *.java`
 - Run `rmic` on implementation class
 - Start `rmiregistry`
 - Start RMI activation daemon
`rmid -J-Djava.security.policy=policyname`
 - Run the setup program
`java -Djava.security.policy=/home/bruce/activedate/policy.all`
`-Djava.rmi.server.codebase=file:/home/bruce/activedate/`
Setup
 - Run the client

Object Activation

- Resources
 - JDK1.3 Documentation
 - <http://java.sun.com/j2se/1.3/docs/guide/rmi/activation.html>